Report

Critical Evaluation of Advanced Algorithms

Introduction

This report evaluates the theoretical complexity and efficiency of algorithms developed across four activities, focusing on optimising Time/Space Complexity and mitigating system bottlenecks.

Activity 1.1: Student Grading System

This task processed student marks to calculate degree classifications, requiring selection of the best 100 credits for Level 5. The system is divided into modular files to separate concerns, ensuring a professional, maintainable code structure.

Data Pre-processing & Time Complexity

An $O(N)$ data cleaning step uses **Pandas** to remove missing values and duplicate rows. To select optimal Level 5 modules, a **Greedy algorithm** sorts modules in descending order at $O(M \log M)$ per student, implemented via Python's `sort()` (Timsort). Dataset iteration uses Pandas `itertuples()`, eliminating object-creation overhead for $O(N)$ loop execution.

Space Complexity & I/O

Space Complexity was heavily optimised to prevent RAM exhaustion. Pandas `inplace=True` flags prevent memory duplication during cleaning. An incremental I/O pipeline via Python's **csv** library processes one `Student` object at a time and flushes it immediately via `writer.writerow()`, keeping memory at a strict $O(1)$ footprint.

Activity 1.2: Brute Force Password Generation

The objective was generating all valid password permutations. The initial randomised $O(L \cdot C^L)$ loop suffered from the *Coupon Collector's* probability problem, mathematically guaranteeing missed combinations.

Time Complexity

The solution was re-engineered into a **Depth-First Search (DFS)** algorithm. While intrinsically $O(C^L)$, constant factors were reduced via **early branch pruning**—instantly terminating recursion paths that violate formatting rules. Built-in **Hash Sets** upgraded constraint validation from $O(N)$ string scans to $O(1)$ lookups.

Space Complexity & I/O

Recursive combination generation risks exponential memory growth, mitigated using a **chunked I/O buffer**. Valid passwords accumulate in RAM then are flushed in batches via `writelines()`, which executes at C-level to minimise system call overhead—balancing fast Disk I/O with a flat $O(L)$ Space Complexity.

Activity 1.3: Railway Route Planning

This task required solving the **Asymmetric Travelling Salesman Problem (ATSP)** across a Directed Graph railway network.

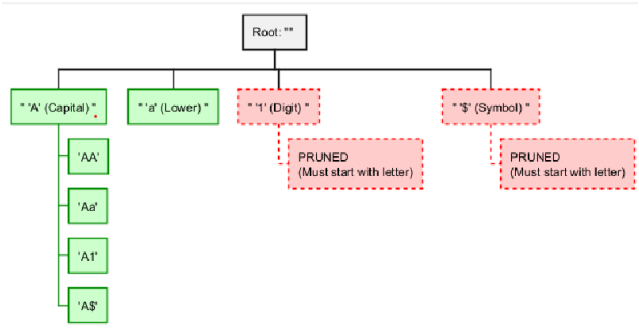

Figure 1: DFS tree showing early branch pruning. Instantly killing recursion paths with invalid starting characters prevents the CPU evaluating millions of impossible permutations.

Time Complexity

Time Complexity

The routing was split into two phases:

1. **Pre-computation:** Dijkstra's algorithm pre-computes a Distance Matrix exclusively for the required waypoints. By utilizing a Min-Heap (Priority Queue) data structure via Python's `heapq` module, the pathfinding achieves an optimal Time Complexity of $O((V + E) \log V)$, where V is the number of stations and E is the number of connections. This avoids the $O(V^2)$ bottleneck of standard array traversals and reduces the main solver to instant $O(1)$ lookups.
2. **Optimisation:** A permutation generator tests all orderings in $O(N!)$ Factorial Time, guaranteeing a mathematically perfect route.

Space Complexity & Library Usage

Python's **itertools** (`itertools.permutations`) operates as a C-optimised generator, yielding one route tuple per iteration rather than building a monolithic array, restricting Space Complexity to $O(N)$.

Beyond Requirements

A custom string-similarity search using partial **Levenshtein** logic dynamically corrects user typos in $O(V)$ time, where V is the number of vertices (stations) in the graph.

Activity 1.4: Facial Recognition

Facial recognition relies on heavy matrix calculations. Sequential processing of high-resolution image directories creates a severe CPU bottleneck.

Architecture: Bypassing the GIL

Standard multithreading cannot bypass Python's Global Interpreter Lock (GIL). `ProcessPoolExecutor` from **concurrent.futures** spawns independent processes, distributing image arrays across all CPU cores for true parallelism. The main process serialises the target encoding matrix and dispatches it via IPC pipes—an implicit **Message Passing model**.

Algorithmic: Early-Exit

Using `as_completed()`, an asynchronous dynamic cancellation algorithm (`pending.cancel()`) terminates all queued images the millisecond a match is confirmed by any worker.

Data-Level: Matrix Reduction

An $O(1)$ pre-processing step via **OpenCV (cv2)** scales unknown images to 25% resolution before detection, reducing evaluated pixels by a factor of 16. Bounding box coordinates are multiplied by 4 to remap to original dimensions.

Library-Level Optimisations

Using the **face_recognition** library, artificial upscaling was disabled (`upsample_num_times=0`) and the faster 5-point facial landmark predictor (`model="small"`) was selected, reducing unnecessary mathematical overhead.

Conclusion

By profiling memory, I/O, and CPU load, the developed algorithms successfully leverage advanced data structures (Hash Sets, Generators, Process Pools) and algorithmic paradigms (Greedy, DFS, Dijkstra) to bypass hardware limitations, achieving superb algorithmic efficiency across all activities.

References

[1] University of the West of England (2023) *Academic Regulations 2023/24*. Bristol: UWE.

[2] Python Software Foundation (2024) *concurrent.futures — Launching parallel tasks*. [online]. Available from: `https://docs.python.org/3/library/concurrent.futures.html` [Accessed 28 February 2026].

[3] Python Software Foundation (2024) *itertools — Functions creating iterators for efficient looping*. [online]. Available from: `https://docs.python.org/3/library/itertools.html` [Accessed 28 February 2026].

[4] OpenCV (2024) *Image Filtering: cv::resize*. [online]. Available from: `https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html` [Accessed 28 February 2026].